50. Почему глобальные переменные – зло?

Если вы попросите ветерана-программиста дать один дельный совет о программировании, то после некоторого раздумья он ответит: «Избегайте использования глобальных переменных!». И, частично, он будет прав. **Глобальные переменные** являются одними из самых злоупотребляемых объектов в языке C++. Хоть они и выглядят безвредными в небольших программах, использование их в крупных проектах зачастую чрезвычайно проблематично.

Новички часто используют огромное количество глобальных переменных, потому что с ними легко работать, особенно когда задействовано много функций. Это плохая идея. Многие разработчики считают, что неконстантные глобальные переменные вообще не следует использовать!

Но прежде, чем мы разберемся с вопросом «Почему?», нужно кое-что уточнить. Когда разработчики говорят, что глобальные переменные — это зло, они НЕ подразумевают полностью ВСЕ глобальные переменные. Они говорят о неконстантных глобальных переменных.

Оглавление:

1. Почему (неконстантные) глобальные переменные — это зло?
2. В чём плюсы использования (неконстантных) глобальных переменных?
3. Как защититься от «глобального разрушения»?
4. Шутка
5. Заключение

Почему (неконстантные) глобальные переменные — это зло?

Безусловно, причина №1, почему неконстантные глобальные переменные являются опасными, — это то, что их значения могут изменять любые вызываемые функции, при этом вы можете этого и не знать. Например, рассмотрим следующую программу:

```
#include <iostream>

// Объявление глобальной переменной
int g_mode;

void doSomething()
{
    g_mode = 2; // присваиваем глобальной переменной g_mode значение 2
}

int main()
{
    g_mode = 1; // примечание: здесь мы присваиваем глобальной переменной g_mode значение 1. Это не объявление локальной перем

    doSomething();

    // Программист по-прежнему ожидает, что g_mode будет 1.
    // Но функция doSomething() изменила значение этой переменной на 2!

    if (g_mode == 1)
        std::cout << "No threat detected.\n";
    else
        std::cout << "Launching nuclear missiles...\n";

    return 0;
}
```

Результат выполнения программы:

```
Launching nuclear missiles...
```

Сначала мы присваиваем переменной `g_mode` значение `1`, а затем вызываем функцию doSomething(). Если бы мы не знали заранее, что doSomething() изменит значение `g_mode`, то, вероятно, не ожидали бы дальнейшего развития событий (`g_mode = 2 => Launching nuclear missiles...`)!

Неконстантные глобальные переменные делают каждую функцию потенциально опасной, и программист не может заранее знать, какая из используемых им функций является опасной, а какая — нет. Локальные переменные намного безопаснее, потому что другие функции не могут влиять на них напрямую.

Также есть много других веских причин не использовать неконстантные глобальные переменные. Например, нередко можно встретить примерно следующее:

```
void boo()
{
    // Некоторый код

    if (g_mode == 4) // делаем что-нибудь полезное
}
```

Предположим, что `g_mode` равно `3`, а не `4` — наша программа выдаст неверные результаты. Как это исправить? Нужно будет отыскать все места, где предположительно могло измениться значение переменной `g_mode`, а затем проследить ход выполнения кода в каждом потенциально опасном участке. Возможно, изменение глобальной переменной вы обнаружите вообще в другом коде, который, как вам казалось на первый взгляд, никак не связан с фрагментом, приведенным выше.

Одной из причин объявления **локальных переменных** максимально близко к месту их первого использования является уменьшение количества кода, которое нужно будет просмотреть, чтобы понять, что делает (зачем нужна?) переменная. С глобальными переменными дела обстоят несколько иначе — поскольку их можно использовать в любом месте программы, то вам придется просмотреть чуть ли не весь код, чтобы проследить логику выполнения и изменения значений переменных в вашей программе.

Например, вы можете обнаружить, что на `g_mode` ссылаются 442 раза в вашей программе. Если использования переменной `g_mode` не подкреплены **комментариями**, то вам придется просмотреть каждое упоминание `g_mode`, чтобы понять, как оно используется в разных случаях.

Также глобальные переменные делают вашу программу менее модульной и гибкой. Функция, которая использует только свои параметры и не имеет **побочных эффектов**, является идеальной в плане модульности. Модульность помогает понять структуру вашей программы, что она делает и как можно повторно использовать определенные участки кода в другой программе. Глобальные переменные значительно уменьшают эту возможность.

В частности, не используйте глобальные переменные в качестве важных переменных, которые выполняют главные или решающие функции в программе (например, переменные, которые используются в условных стейтментах, как `g_mode` выше). Ваша программа вряд ли сломается, если в ней будет глобальная переменная с информационным значением, которое может меняться (например, имя пользователя). Гораздо хуже, если изменится значение глобальной переменной, которая влияет непосредственно на результаты выполнения самой программы или на её работу.

Правило: Вместо глобальных переменных используйте локальные (когда это целесообразно).

В чём плюсы использования (неконстантных) глобальных переменных?

Их немного. Зачастую можно обойтись без использования неконстантных глобальных переменных. Но в некоторых случаях их разумное использование поможет уменьшить сложность программы, и, иногда, может быть даже лучшим вариантом для решения проблемы, чем альтернативные способы.

Например, если ваша программа использует базу данных для чтения и записи данных, то имеет смысл определить базу данных глобально, поскольку доступ к ней может понадобиться с любого места. Аналогично, если в вашей программе есть журнал ошибок (или журнал **отладки**), в котором вы можете читать/записывать информацию об ошибках (или об отладке), то имеет смысл определить его глобально. Звуковая библиотека может быть еще одним хорошим примером: вам, вероятно, не захочется открывать доступ к ней для каждой функции, которая делает запрос. Поскольку у вас будет только одна звуковая библиотека, управляющая всеми звуками, логично будет объявить её глобально, инициализировать при запуске программы, а затем использовать только в режиме чтения.

Как защититься от «глобального разрушения»?

Если у вас возникнет ситуация, где полезнее будет использовать неконстантные глобальные переменные, вместо локальных, то вот вам несколько полезных советов, которые помогут свести к минимуму количество потенциальных проблем, с которыми вы можете столкнуться при использовании подобных переменных.

Во-первых, добавляйте префикс `g_` ко всем вашим глобальным переменным и/или размещайте их в пространстве имен, дабы уменьшить вероятность возникновения **конфликтов имен**.

Например, вместо следующего:

```
#include <iostream>

double gravity (9.8); // по имени переменной непонятно, глобальная ли это переменная или локальная

int main()
{
    return 0;
}
```

Сделайте следующее:

```
#include <iostream>

double g_gravity (9.8); // теперь понятно, что это глобальная переменная

int main()
{
    return 0;
}
```

Во-вторых, вместо разрешения прямого доступа к глобальным переменным, лучше их «инкапсулировать». Сначала добавьте ключевое слово `static`, чтобы доступ к ним был возможен только из файла, в котором они объявлены. Затем напишите внешние глобальные «функции доступа» для работы с переменными. Эти функции помогут обеспечить надлежащее использование переменных (например, при проверке ввода, проверке допустимого диапазона значений и т.д.). Кроме того, если вы когда-либо решите изменить первоначальную реализацию программы (например, перейти из одной базы данных в другую), то вам нужно будет обновить только *функции доступа* вместо каждого фрагмента кода, который напрямую использует глобальные переменные.

Например, вместо следующего:

```
double g_gravity (9.8); // можно экспортировать и использовать напрямую в любом файле
```

Сделайте следующее:

```
static double g_gravity (9.8); // ограничиваем доступ к переменной только на этом файл

double getGravity() // эта функция может быть экспортирована в другие файлы для доступа к глобальной переменной
{
    return g_gravity;
}
```

В-третьих, при написании автономной функции, использующей глобальные переменные, не используйте их непосредственно в теле функции. Передавайте их в качестве параметров. Таким образом, если в вашей функции нужно будет когда-либо использовать другое значение, то вы сможете просто изменить параметр. Это улучшит модульность вашей программы.

Вместо следующего:

```
// Эта функция полезна только для расчета мгновенной скорости на основе глобальной гравитации
double instantVelocity(int time)
{
    return g_gravity * time;
}
```

Сделайте следующее:

```
// Эта функция вычисляет мгновенную скорость для любого значения гравитации.
// Передайте возвращаемое значение из getGravity() в параметр gravity, если хотите использовать глобальную переменную gravity
double instantVelocity(int time, double gravity)
{
    return gravity * time;
}
```

Наконец, изменение значений глобальных переменных — это прямой путь к проблемам. Структурируйте ваш код в соответствии с тем, что ваши глобальные переменные могут измениться. Постарайтесь свести к минимуму количество случаев, где они могут изменять свои значения — обращайтесь с ними исключительно как с *доступными только для чтения* (насколько позволяет ситуация). Если вы можете инициализировать значение глобальной переменной при запуске программы, а затем не изменять его в ходе выполнения, то, таким образом, вы снизите вероятность возникновения непредвиденных проблем.

Шутка

Какой наилучший префикс для глобальных переменных?

Ответ: `//`.

Заключение

Избегайте использования неконстантных глобальных переменных, насколько это возможно! Если же используете, то используйте их максимально разумно и осторожно.